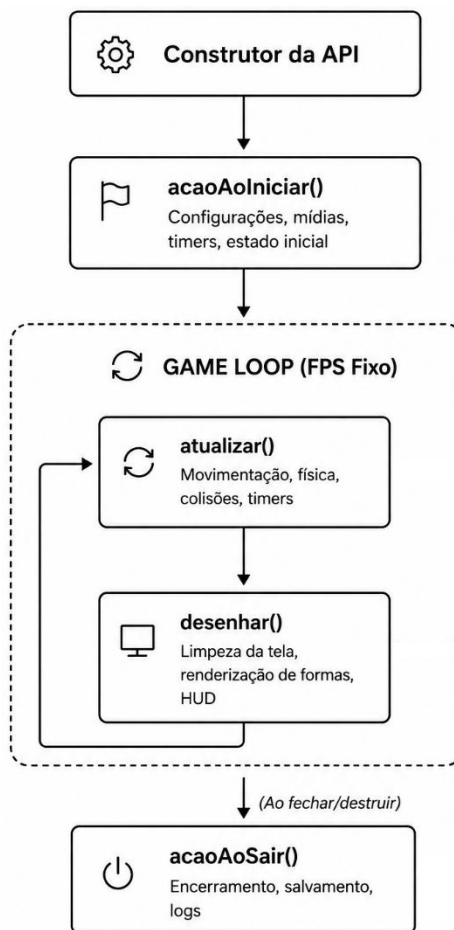


JS_CG_2D_API – Apresentação

A JS_CG_2D_API é uma biblioteca didática desenvolvida para simplificar a criação de jogos e simulações 2D no HTML5 Canvas em JavaScript, encapsulando o gerenciamento do loop do jogo (*Game Loop*), renderização gráfica, controle de entradas (teclado/mouse/touch), áudio e temporização.

Arquitetura e Ciclo de Vida

A API é baseada no padrão de herança orientada a objetos. Sua classe de jogo deve estender JS_CG_2D_API e sobrescrever os métodos do ciclo de vida.



Modelo de Instanciação

```
new MeuJogo("Título da Janela", "idDoCanvasHTML", largura, altura);  
// Exemplo:  
new MeuJogo("Space Defender", "gameCanvas", 800, 600);
```

A seguir, são apresentados os detalhes de cada componente da API, para um melhor entendimento do seu completo funcionamento.

1. Módulo Auxiliar: Enumeração Estilo

Objeto congelado (Object.freeze) que define como as primitivas geométricas serão desenhadas na tela.

PROPRIEDADE	VALOR	DESCRIÇÃO
Estilo.PONTOS	1	Desenha apenas os vértices (pontos) da forma geométrica.
Estilo.LINHAS	2	Desenha apenas o contorno (<i>wireframe</i>) da forma geométrica.
Estilo.PREENCHIDO	3	Preenche a forma com a cor definida em preenchimento().

2. Módulo Auxiliar: Classe Retangulo2D

Representa uma caixa delimitadora alinhada aos eixos (**AABB** - *Axis-Aligned Bounding Box*). Utilizada para colisão e posicionalmente por vários métodos da API.

Instanciação

```
let caixa = new Retangulo2D(x, y, largura, altura);
```

Propriedades e Métodos

- **this.x / this.y**: Coordenadas da aresta superior esquerda.
- **this.largura / this.altura**: Dimensões da caixa.
- **Getters AABB**: minX, minY, maxX, maxY (disponíveis como getters JS e métodos como getMinX(), getMaxX(), etc.).
- **intersects(outro)**: Retorna true se houver sobreposição com outra instância de Retangulo2D.

```
let blocoA = new Retangulo2D(10, 10, 50, 50);
let blocoB = new Retangulo2D(40, 40, 50, 50);

if (blocoA.intersects(blocoB)) {
  console.log("Colisão AABB detectada!");
}
```

3. Módulo Auxiliar: Classe Estática EfeitosSonoros

Gerenciador estático global de áudio baseado na API HTML5 Audio. Não requer instanciação.

Métodos Estáticos

- **EfeitosSonoros.carregarSom(id, src)**

Registra um áudio na memória associando a uma chave string.

```
EfeitosSonoros.carregarSom("pulo", "sons/jump.wav");
```

- **EfeitosSonoros.volumeSom(id, volume)**

Ajusta o volume do áudio. O parâmetro volume varia de 0.0 (mudo) a 1.0 (volume máximo).

```
EfeitosSonoros.volumeSom("pulo", 0.5); // 50% do volume
```

- **EfeitosSonoros.tocarSom(id, exclusivo, sobreposto)**

Executa o som correspondente ao id.

- **exclusivo** (*boolean, padrão = false*): Se true, pausa e reinicia todos os outros sons antes de tocar este.
- **sobreposto** (*boolean, padrão = false*): Se true, clona o elemento de áudio no momento do disparo. **Essencial para tiros e coletas rápidas**, permitindo que o mesmo som toque várias vezes simultaneamente sem cortar a execução anterior.

```
// Toca permitindo sobreposição rápida (ex: tiro rápido)
EfeitosSonoros.tocarSom("tiro", false, true);

// Toca interrompendo todas as músicas/efeitos de fundo (ex: Game Over)
EfeitosSonoros.tocarSom("gameover", true, false);
```

- **EfeitosSonoros.pararSom(id)**

Interrompe a reprodução e reseta o ponteiro de execução para o início.

4. Módulo Auxiliar: Classe Personagem

Motor físico básico para jogos de plataforma. Controla a gravidade, aceleração, velocidade terminal e colisão vertical e horizontal com plataformas.

Instanciação

```
let heroi = new Personagem(x, y, largura, altura, [arrayDePlataformas]);
```

Atributos e Configurações

- **gravidade**: Aceleração aplicada no eixo Y por frame (padrão: 0.6).
- **forçaPulo**: Impulso vertical (padrão: -12.0). Deve ser **negativo** para mover o personagem para cima.
- **velTerminal**: Velocidade máxima de queda livre (padrão: 18.0).
- **noChao**: Booleano que indica se a entidade está apoiada em uma plataforma.

Métodos

- **atualizar()**: Aplica a gravidade, atualiza y e resolve a colisão física com a lista de plataformas atreladas. Deve ser chamado no atualizar() do jogo.
- **pular()**: Dispara o pulo **apenas** se noChao for verdadeiro.
- **getColisor()**: Retorna a instância de Retangulo2D atualizada do personagem para checagens adicionais.
- **addPlataformas(p)**: Adiciona uma plataforma individual ou um Array de plataformas.

```
// Exemplo de uso no jogo
this.plataformas = [ new Retangulo2D(0, 500, 800, 40) ];
this.jogador = new Personagem(100, 100, 32, 48, this.plataformas);

// No método atualizar()
this.jogador.atualizar(); // Aplica gravidade e resolve colisões
if (this.teclas["Space"]) this.jogador.pular();
```

5. Módulo Auxiliar: Classe Sprite

Entidade visual animada composta por quadros de imagens (HTMLImageElement). Os quadros de imagens são obtidos normalmente através da extração de quadros de imagens do tipo GIF, salvos em PNG. Para isso, pode-se utilizar o website <https://ezgif.com/split> que fornece ferramenta on-line gratuita para extração de quadros do GIF.

Instanciação

```
let inimigo = new Sprite(x, y);
```

Propriedades e Métodos

- **px / py**: Posição na tela.
- **largura/altura**: largura e altura da imagem do Sprite (pode ser definida no construtor, ou setada via método **setTamanho(novaLargura, novaAltura)**).

- **vx / vy**: Velocidades no vetor X e Y.
- **delayAnimacao**: Quantidade de quadros do jogo necessários para avançar 1 frame de animação (padrão: 6). Quanto menor, mais rápida a animação.
- **setAnimacao(listaImagens)**: Modifica o array de frames da animação atual e reseta o contador.
- **getImagem()**: Retorna o HTMLImageElement ativo no frame atual.

6. Módulo Auxiliar: Classe BotaoTouch

Entidade para criar controles virtuais na tela. Botões virtuais são desenhados automaticamente ao final de cada frame, e podem simular teclas do teclado e acionar a mesma lógica já usada no desktop. Possui suporte a múltiplos toques (múltiplos botões tocados simultaneamente), e o mesmo botão também funciona no Desktop.

Criando Botões

```
// Os botões utilizam as mesmas teclas tratadas
// por teclaPressionada() e teclaLiberada().
this.criarBotaoTouch("esquerda", 35, 480, 75, 65, "◀", "ArrowLeft");
this.criarBotaoTouch("direita", 125, 480, 75, 65, "▶", "ArrowRight");
this.criarBotaoTouch("pular", 690, 480, 75, 65, "▲", " ");
```

Um Botão Touch pode ser personalizado:

- setCores(corPadrao, corPressionado, corBorda, corTexto)
- setCorPadrao(cor)
- setCorPressionado(cor)
- setCorBorda(cor)
- setCorTexto(cor)
- setTamanhoFonte(tamanho)
- setVibracao(ms)

Para usar os botões, normalmente usa-se a tecla associada, assim é tratado no evento de teclado normalmente:

```
teclaPressionada(e) {
  if (e.key === "ArrowLeft") {
    this.esquerda = true;
  }
  if (e.key === "ArrowRight") {
    this.direita = true;
  }
  if (e.key === " ") {
    this.jogador.pular();
  }
}
```

```
teclaLiberada(e) {
  if (e.key === "ArrowLeft") {
    this.esquerda = false;
  }
  if (e.key === "ArrowRight") {
    this.direita = false;
  }
}
```

Quando não se associa uma tecla ao botão, então deve-se usar evento do mouse para capturar a ação, e verificar com o método **foiClicado(e)**:

```
this.criarBotaoTouch("btnPause", 400, 20, 90, 40, "PAUSE")
  .setCores(
    "■ rgba(13, 110, 253, 0.6)",
    "■ rgba(10, 70, 160, 0.9)",
    "■ rgba(147, 197, 253, 0.8)",
    "■ #FFFFFF",
  )
  .setTamanhoFonte(16)
  .setVibracao(30);
```

```
cliqueDoMouse(e) {
  let btnPausa = this.getBotaoTouch("btnPause");
  if (btnPausa && btnPausa.foiClicado(e)) {
    this.jogoPausado = !this.jogoPausado;
  }
}
```

7. Classe Principal: JS_CG_2D_API

A classe base que controla a engine gráfica, janelas, contexto do Canvas, entradas, loop do jogo e renderizações.

Configuração do Construtor

```
constructor(nome, canvasId, w, h, fps = 60)
```

Se o elemento <canvas> com o canvasId informado não existir no documento HTML, a API cria o elemento dinamicamente na página.

Como sugestão, no modelo padrão de html adotado para criar o jogo não cria-se o elemento canvas, deixando a própria API cria-lo (conforme própria documentação interna da API). Caso queira criar um elemento manualmente também é válido, porém deve-se atribuir um id ao mesmo e passar esse id para o construtor do jogo.

Exemplo de HTML padrão para um jogo:

```
<!DOCTYPE html>
<meta charset="UTF-8">
<script src="js_cg_2d_api.js"></script>
<script src="jogo.js"></script>
```

A. Ciclo de Vida e Manipuladores de Eventos

Métodos projetados para serem **sobrescritos** pela sua classe de jogo:

```
class MeuJogo extends JS_CG_2D_API {  
    acaoAoIniciar() { /* Roda 1x antes de iniciar o Game Loop */ }  
    atualizar()      { /* Roda N vezes por segundo (lógica/física) */ }  
    desenhar()      { /* Roda N vezes por segundo (gráficos) */ }  
    acaoAoSair()    { /* Roda ao fechar/recarregar a aba */ }  
  
    // Eventos de Teclado  
    teclaPressionada(e) { /* e.code ou e.key */ }  
    teclaLiberada(e)   { /* e.code ou e.key */ }  
  
    // Eventos de Mouse  
    cliqueDoMouse(e)      { /* e.x, e.y */ }  
    movimentoDoMouse(e)   { /* e.x, e.y */ }  
    movimentoDoMousePressionado(e) { /* Chamado durante o arrasto */ }  
    mousePressionado(e)   { /* Down */ }  
    mouseSolto(e)         { /* Up */ }  
}
```

Métodos do Ciclo de Vida

- **acaoAoIniciar():** Chamado uma única vez na criação do jogo. É responsável por carregar sons, inicializar pontuações, criar variáveis, iniciar timers.
- **atualizar(dt):** Executado repetidamente (frequência definida pelo FPS). É responsável por toda a lógica do jogo: física, movimentação, cálculo de colisões e pontuação.
 - **dt** é um parâmetro opcional, significa **Delta Time** (tempo decorrido desde o último frame em segundos). Permite criar movimentações independentes da taxa de FPS (ex: velocidade * dt).
- **desenhar():** Executado a cada frame após o atualizar(). É responsável apenas pela renderização gráfica: desenhar fundos, sprites, formas e HUD.
- **acaoAoSair():** Chamado ao encerrar o jogo. É responsável por exibir mensagens de game over, logs de estatísticas ou salvar dados.

OBS: Nunca altere regras do jogo (como pontos ou vida) nem instancie novos objetos pesados dentro do método desenhar(). Isso causa queda massiva de desempenho (*memory leak*) e comportamento imprevisível no motor.

Métodos de Entrada (Teclado e Mouse)

A API mapeia nativamente os eventos do navegador para métodos sobrescrevíveis.

Teclado

- **teclaPressionada(e)**: Disparada ao pressionar uma tecla.
- **teclaLiberada(e)**: Disparada ao soltar uma tecla.
 - **Leitura recomendada**: Utilize `e.code` ("ArrowUp", "KeyW", "Space") ou `e.key`.
 - **Tela Cheia**: É uma boa prática ativar com `if (e.altKey && e.code === "KeyF") this.telaCheia();`.

Mouse

- **cliqueDoMouse(e)**: Ocorre ao clicar dentro do canvas.
- **movimentoDoMouse(e)**: Ocorre ao mover o cursor sobre o canvas.
- **movimentoDoMousePressionado(e)**: Ocorre durante o arrasto.
- **mousePressionado(e)**: Ocorre quando o botão do mouse foi pressionado.
- **mouseSolto(e)**: Ocorre quando o botão do mouse for liberado.
 - **Captura de Posição**: Utilize `e.x` e `e.y` (coordenadas locais dentro do canvas).
 - **Quando usar *cliqueDoMouse* ou *mousePressionado***: Pense no botão de atirar num jogo de tiro vs. o botão de comprar uma arma na Loja.
 - Para **atirar**, se esperar o jogador soltar o dedo do mouse, o inimigo já andou. Para resposta imediata: **mousePressionado**.
 - Para **comprar**, se encostar o dedo por engano e não puder desistir arrastando o mouse para fora, vai gastar dinheiro sem querer. Para confirmação: **cliqueDoMouse**.

B. Primitivas Gráficas e Estilização

Antes de desenhar qualquer forma, é necessário definir a cor do **preenchimento** ou do **contorno**.

Estilização

- **preenchimento(cor)**: Define a cor atual para preenchimentos internos ("red", "#00FF00", "rgba(0,0,0,0.5)").
- **contorno(espessuraOuCor, corOpcional)**: Define estilo de linhas e contornos.
- **Estilo.PREENCHIDO** e **Estilo.LINHAS**: Constantes passadas às formas para definir o modo de renderização.

```
this.contorno("blue"); // Mantém a espessura e altera a cor  
this.contorno(4, "yellow"); // Define espessura de 4px e cor amarela
```

Métodos de Desenho

- **limparTela(cor)**: Preenche todo o canvas com a cor informada.
- **ponto(x, y, tam)**: Desenha um ponto centralizado nas coordenadas informadas.

- **linha(xi, yi, xf, yf, estilo):** Desenha um segmento de reta entre \$(xi, yi)\$ e \$(xf, yf)\$.
- **retangulo(x, y, l, a, estilo):**
 - Aceita 5 parâmetros numéricos (x, y, largura, altura, estilo).
 - **OU** aceita um objeto Retangulo2D e o estilo: `this.retangulo(meuRetangulo2D, Estilo.PREENCHIDO)`.
- **circulo(x, y, l, a, estilo):** Desenha um círculo/elipse contido dentro do retângulo delimitador de posição (x,y) e dimensões (l,a).
- **triangulo(x0, y0, x1, y1, x2, y2, estilo):** Desenha um triângulo conectando os 3 vértices.
- **paralelogramo(x0, y0, x1, y1, x2, y2, x3, y3, estilo):** Desenha um quadrilátero conectando 4 vértices.
- **poligono(vetX, vetY, estilo):** Conecta sequencialmente os pontos definidos pelos arrays paralelos de vértices vetX e vetY.
- **imagem(imgObj, x, y, l, a):** desenha uma imagem previamente carregada, na posição x, y, podendo definir largura e altura da mesma.
- **texto(conteudo, x, y, tamanho, tipo):** Escreve o texto informado.

```
this.texto("Vidas: 3", 20, 50, 24, "bold");
```

C. Transformações Geométricas 2D (Pilha de Matrizes)

Permitem mover, girar e escalar a origem do sistema de coordenadas gráfico local.

- **empilhar():** Salva a matriz de transformação e as cores/estilos atuais (save).
- **desempilhar():** Restaura o estado salvo anteriormente (restore).
- **transladar(x, y):** Move a origem do plano (0,0) para a posição (x,y).
- **rotacionar(ang):** Rotaciona o plano de coordenadas em **graus** (a API converte internamente para radianos).
- **escalar(x, y):** Aplica escala nos eixos X e Y.

```
// 1. Guardar o checkpoint do mundo (ponto 0,0 no canto superior esquerdo)
this.empilhar();
// 2. Mover o centro do mundo para a posição do carro
this.transladar(this.carroX, this.carroY);
// 3. Girar a mesa de desenho no ângulo atual
this.rotacionar(this.angulo);
// 4. Desenhar o carro centralizado no próprio centro (0,0)
// Um retângulo de 30x20 desenhado de -15 a +15 fica perfeitamente centralizado!
this.preenchimento("blue");
this.retangulo(-15, -10, 30, 20, Estilo.PREENCHIDO);
// Um pequeno detalhe (farol) para indicar a frente do carro
this.preenchimento("yellow");
this.retangulo(10, -8, 5, 16, Estilo.PREENCHIDO);
// 5. Restaurar o mundo para a posição original
this.desempilhar();
```

- Para as transformações usamos o conceito de "**Mover a folha de papel**" (Sistema de Coordenadas Local). A intuição de "desenhar primeiro e depois mover" parece natural para um objeto físico, mas em Computação Gráfica o motor gráfico ajusta a **mesa de trabalho (o sistema de coordenadas)** *antes* de rabiscar. A CPU executa linha por linha (**de cima para baixo**):
- **1. empilhar()** — "**Guardar um checkpoint do mundo**": Salva a posição original da tela (onde o ponto $(0,0)$ fica no canto superior esquerdo).
 - **2. transladar(x, y)** — "**Mover o centro do mundo**": Desloca a origem $(0,0)$ do mapa diretamente para a posição atual do carro na tela.
 - **3. rotacionar(ang)** — "**Girar a mesa de desenho**": Gira todo o papel em volta desse novo ponto $(0,0)$.
 - **4. retangulo(-15, -10, ...)** — "**Desenhar focado no próprio umbigo**": Agora que a mesa já está no lugar e rotacionada, o carro se desenha **centralizado na sua própria origem $(0,0)$** .
 - *Por que -15, -10?* Porque um retângulo de tamanho 30×20 desenhado de -15 a +15 fica perfeitamente centralizado no ponto de rotação!
 - **5. desempilhar()** — "**Restaurar o mundo**": Volta o papel para a posição original, para que o próximo elemento do jogo não seja desenhado torto ou fora do lugar.

D. Sistema Unificado de Colisão (colisao)

A API unifica os testes de interseção **AABB** (caixa delimitadora alinhada aos eixos) em um único método flexível:

1. **Através de 8 Coordenadas Numéricas:**

```
this.colisao(x1, y1, w1, h1, x2, y2, w2, h2);
```

2. **Através de Dois Objetos:** O método extrai automaticamente caixas delimitadoras de instâncias de Retangulo2D, Personagem, Sprite ou objetos genéricos que contenham $\{x, y, largura, altura\}$ ou $\{x, y, w, h\}$.

```
if (this.colisao(this.jogador, this.moeda)) {  
    // ...  
}
```

E. Gerenciamento de Temporizadores (Timers)

Timers integrados ao Game Loop que utilizam o tempo delta real em segundos (independente de oscilações na taxa de FPS).

- **iniciarTimer(nome, segundos, repetir, callback):**

```
// Dispara a cada 2 segundos continuamente
this.iniciarTimer("spawner", 2.0, true, () => {
  this.criarInimigo();
});
```

- **getTimer(nome):** Retorna o tempo restante em segundos com precisão flutuante. Retorna -1 se o timer expirou ou não existir.
- **pararTimer(nome):** Cancela a contagem do timer.
- **pausarTimer(nome):** pausa a contagem de um timer.
- **retomarTimer(nome):** restaura a execução de um timer.

F. Manipulação de Sprites e Imagens

- **carregarFrames(nomeBase, quantidade, pasta):** Carrega em lote arquivos numerados. Por padrão, procura em imagens/nomeBase/. *Exemplo:* carregarFrames("run", 4) busca imagens/run/run_1.png até run_4.png.
- **desenharSprite(sprite):** Renderiza o frame atual da animação vinculada à instância de Sprite.

7. Configuração de Autocomplete no VSCode

Copiar o arquivo **jsconfig.json** (disponível no diretório **api**) para o diretório **raiz** onde desenvolverá os projetos no **VSCode**, garantindo assim que o autocompletar da api funcione enquanto edita qualquer script em qualquer subdiretório.

- **Extensões utilizadas no VSCode:**
 - Code Runner
 - Open PHP/HTML/JS in Browser
 - Prettier - Code Formatter.